

Lecture 5

Choosing, Adapting & Deploying LLMs

A full-stack engineering guide to evaluating model architectures, domain adaptation, quantization mechanics, and production GPU serving infrastructure.

| Course Agenda & Curriculum



1. Model Selection

Taxonomy, parameter scales, performance metrics, and proprietary vs open-weights tradeoffs.



2. Architecture

Transformer mechanics, tokenization, context windows, and sampling hyperparameters.



3. Adaptation

In-context prompting, RAG architectures, PEFT / LoRA fine-tuning, and alignment (DPO/RLHF).



4. Quantization

Precision reduction (FP32 to INT4), PTQ vs QAT, AWQ/GPTQ, and VRAM memory profiling.



5. Infrastructure

vLLM & TensorRT-LLM serving engines, PagedAttention, KV cache, and GPU selection.



6. Operations

LLMOps lifecycle, evaluation benchmarks, security guardrails, and cost optimization.

MODULE 1

The LLM Landscape & Model Sizing

Selecting the optimal foundational model for your specific enterprise constraints.

| Model Sizing Taxonomy



Small (1B - 8B)

Latency: < 50ms

Use Cases: Routing, intent detection, entity extraction, edge deployment.

Compute: Runs on single consumer GPU or CPU.



Medium (14B - 70B)

Latency: 100ms - 500ms

Use Cases: Enterprise RAG, code generation, multi-step summarization.

Compute: Requires 1-4 enterprise GPUs (A100/H100).



Flagship (70B+)

Latency: 500ms - 2s+

Use Cases: Complex reasoning, autonomous agent planning, scientific analysis.

Compute: Distributed GPU clusters (8+ H100s).

| Proprietary APIs vs. Open-Weights

Dimension	Proprietary APIs (e.g., GPT-4o, Claude 3.5)	Open-Weight Models (e.g., Llama 3, Mistral)
Time-to-Market	Instant access via REST API endpoints.	Requires infrastructure provisioning & deployment setup.
Data Privacy	Data sent to third-party vendor cloud endpoints.	Complete air-gapped data ownership & self-hosting.
Customizability	Limited to prompt engineering & basic API fine-tuning.	Full access to model weights, architecture, and embedding spaces.
Cost Structure	Variable per-token pricing model. Scaling can be costly.	Fixed hardware/GPU infrastructure cost with zero token fees.

| Key Benchmark Evaluation Metrics

MMLU

MULTI-TASK KNOWLEDGE

GSM8K

GRADE SCHOOL MATH

HumanEval

PYTHON CODE GENERATION/FUNCTIONAL
CORRECTNESS

Standardized benchmarks provide initial directional filtering. However, production selection requires domain-specific evaluations (Human-in-the-Loop & LLM-as-a-Judge) tailored to your target data distribution.

MODULE 2

Model Foundations & Architecture

Understanding tokenization, transformer attention mechanisms, and inference dynamics.

| Parameters & Context Windows

70B

MODEL WEIGHT MATRICES

128K

ACTIVE CONTEXT TOKENS

32K

VOCABULARY SIZE (TOKENS)

Parameters: The internal learned neural weights (floating-point values) that encode factual patterns and reasoning heuristics across feed-forward networks.

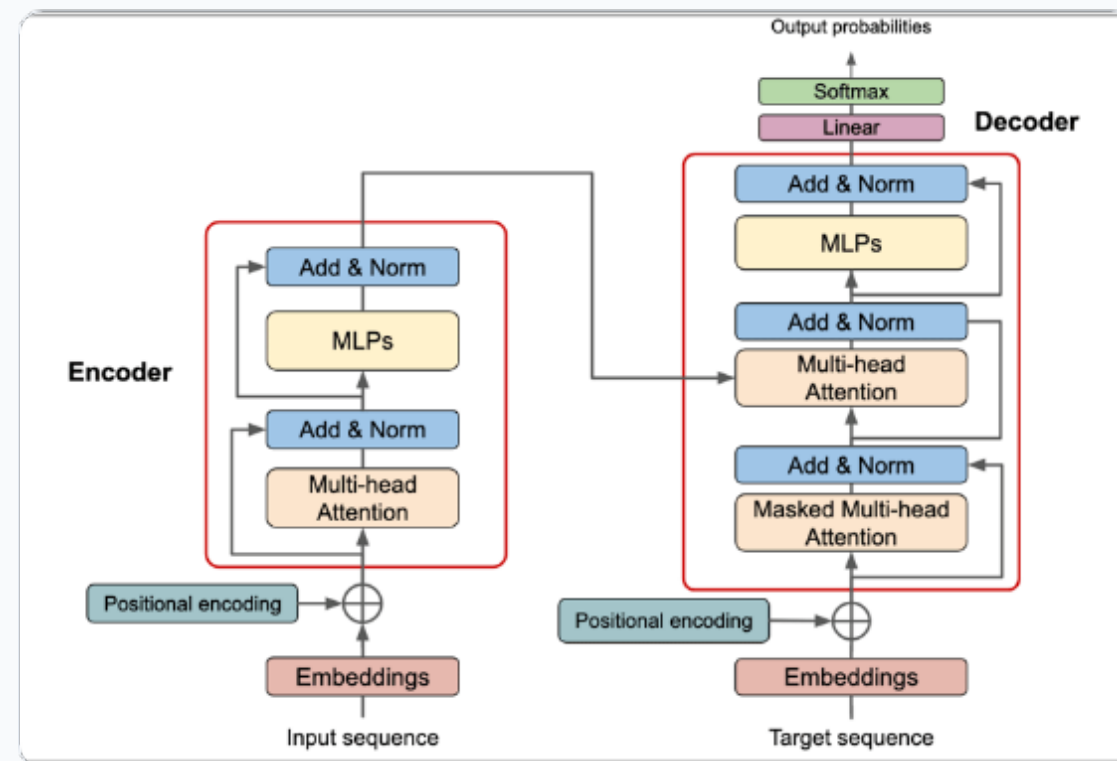
Context Window: The fixed capacity buffer of sequential tokens the self-attention mechanism can process simultaneously without catastrophic recall degradation.

The Transformer Architecture

Decoder-Only Autoregressive Paradigm

Modern causal LLMs utilize stacked decoder blocks with standard components:

- ↔ **Multi-Head Self-Attention:** Projects input tokens into Query, Key, and Value matrices to capture token dependencies.
- ≡ **Feed-Forward Layers (FFN):** Non-linear projections storing world knowledge and factual associations.
- 🛡 **RMSNorm & Residuals:** Stabilize gradient flow across deep multi-layer stacks (e.g., 80+ layers).



| Inference Sampling Hyperparameters



Temperature

Scales logit logits prior to softmax. Lower values (0.0) yield deterministic outputs; higher values (>0.7) increase tail-token entropy.



Top-p (Nucleus)

Samples from the smallest cumulative set of tokens whose probabilities sum to p (e.g., 0.9), filtering low-probability noise dynamic per step.



Top-k

Truncates sampling strictly to the top k highest-probability tokens regardless of overall cumulative probability distribution mass.

| Next-Token Probability Math

 Autoregressive models compute probability over sequence tokens via softmax normalization.

 Objective loss minimization during pretraining utilizes standard Cross-Entropy Loss:

Cross-Entropy Loss Formulation

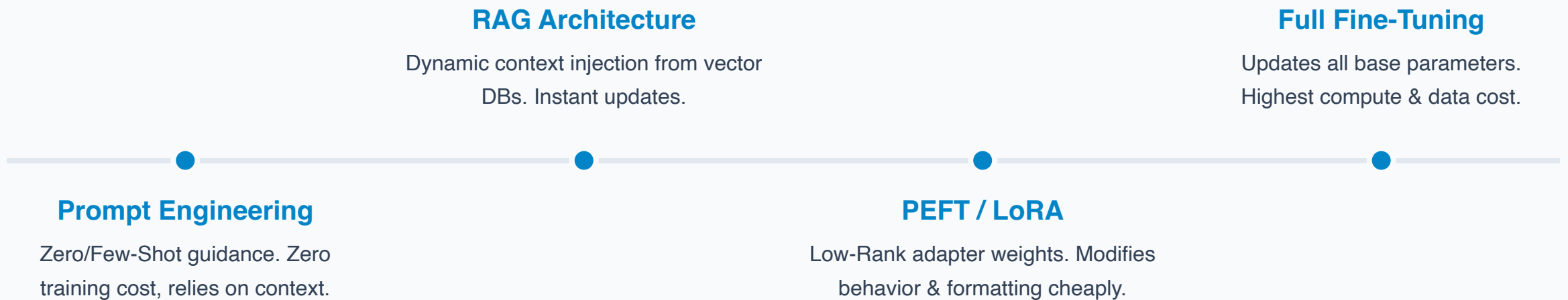
$$L = - \sum_{t=1}^T \log P (x_t \mid x_1 , x_2 , \dots , x_{t-1})$$

MODULE 3

Adaptation & Customization Strategies

From In-Context Prompting to Retrieval-Augmented Generation and Parameter-
Efficient Fine-Tuning.

| The Spectrum of Model Adaptation



| In-Context Learning Patterns

-  **System Prompting & Role Alignment:** Encapsulate behavioral boundaries, output format specifications (JSON Schema), and safety guidelines.
-  **Chain-of-Thought (CoT):** Explicitly prompt the model to emit step-by-step intermediate reasoning paths prior to final answer generation.
-  **Few-Shot Exemplars:** Provide 3–5 representative input/output pairs directly inside the context window to enforce rigid structural schema compliance.

RAG Architecture

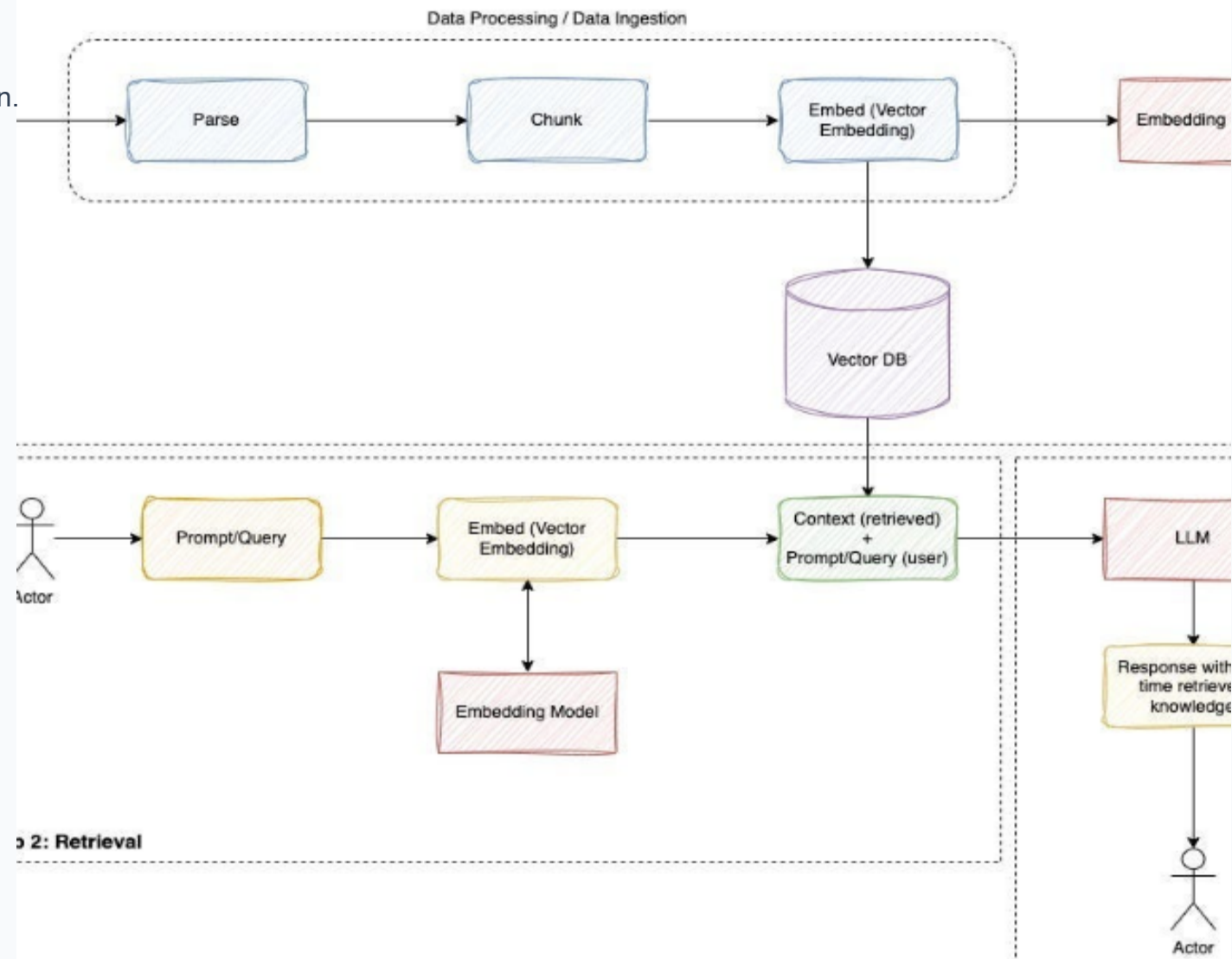
Retrieval-Augmented Generation

Decouples factual knowledge from static model parameters by retrieving relevant enterprise document chunks dynamically during query generation.

Prevents hallucinations, maintains strict access control, and eliminates recurring full-model retraining pipelines.

Retrieval Augment Generation (RAG)

Vector Database



| Fine-Tuning: Full vs. LoRA (PEFT)

Full Parameter Tuning

Updates 100% of base model weights. Requires massive GPU clusters to store optimizer states (e.g., AdamW 8x VRAM multiplier). Risky due to catastrophic forgetting.

Low-Rank Adaptation (LoRA)

Freezes base weights and injects trainable rank-decomposition matrix pairs into attention layers. Reduces trainable parameters by >99% while achieving matching performance.

RAG vs. Fine-Tuning Matrix

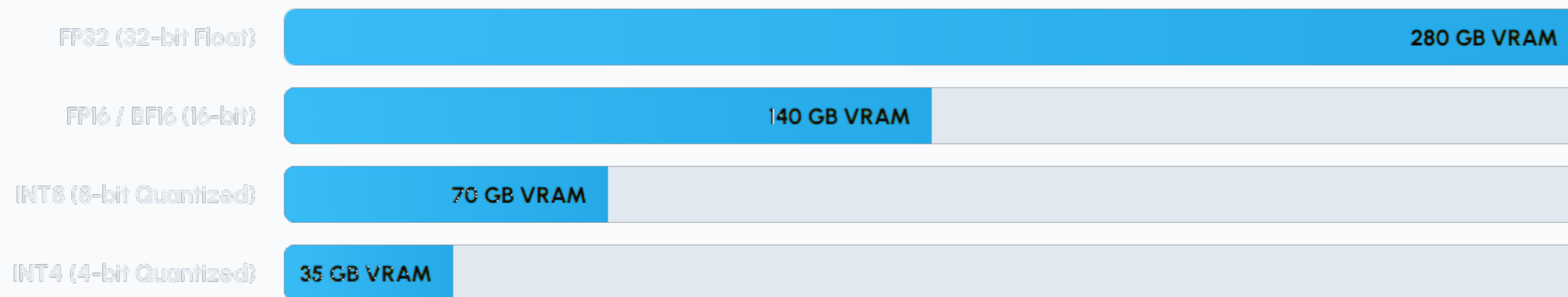
Requirement / Dimension	Retrieval-Augmented Generation (RAG)	Fine-Tuning (LoRA / SFT)
Primary Objective	Inject fresh, domain-specific facts & documents.	Teach tone, style, specific output syntax/format.
Knowledge Updating	Instant (update vector DB index in seconds).	Slow (requires dataset prep & re-training run).
Hallucination Control	High (verifiable source attribution via context).	Moderate (model still prone to confident guessing).
Context Window Footprint	Consumes context with retrieved chunks.	Zero extra prompt tokens required for style/format.

MODULE 4

Model Compression & Quantization

Fitting multi-billion parameter models into production GPU memory footprints.

| VRAM Footprint Across Numerical Precision



Memory required strictly for storing model weights of a 70B parameter model. KV Cache & activations add +15–20% overhead.

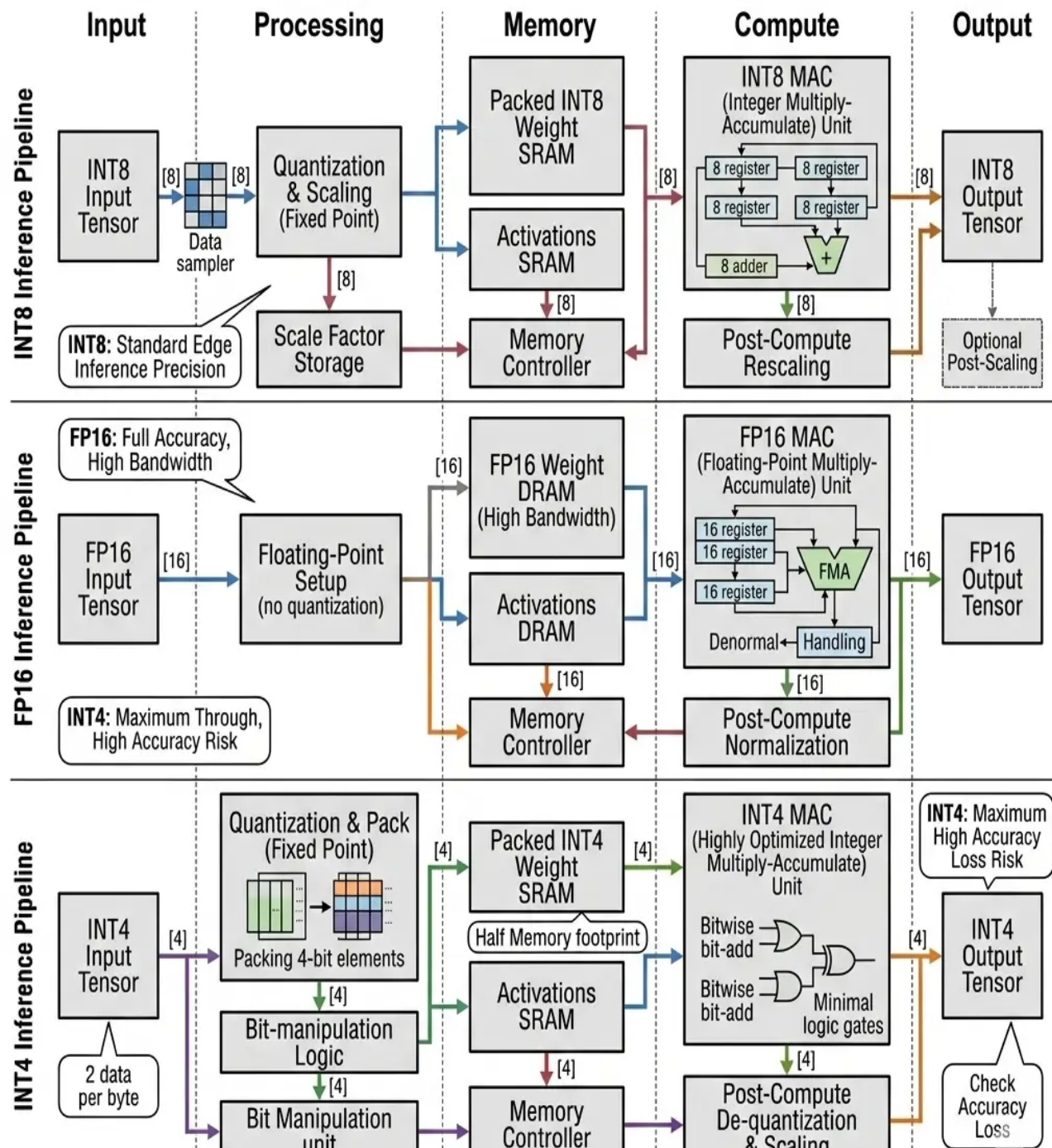
Quantization Mechanics

PTQ vs Outlier Protection

Post-Training Quantization (PTQ) maps continuous floating-point weight distributions into discrete integer bins.

Advanced algorithms like AWQ & GPTQ identify and preserve critical outlier weights in 16-bit precision, preventing degradation in model reasoning.

EXECUTION PIPELINE COMPARISON: INT8 vs FP16 vs INT4 INFERENCE



| Alternative Compression Strategies

Sparse Pruning

Removes redundant network connections or entire attention heads that contribute minimally to loss. Structured pruning accelerates hardware execution speed directly.

Knowledge Distillation

Trains a smaller "Student" model (e.g., 8B) to mimic the probability logit distribution of a massive "Teacher" model (e.g., 405B), transferring implicit reasoning.

VRAM Budget Estimation Matrix

Memory Component	Estimation Formula	70B Model Example (FP16)
Model Weights	Parameters × Bytes per Precision	70B × 2 Bytes = 140 GB
KV Cache Allocation	2 × Layers × Heads × Head_Dim × Seq_Len × Batch	8192 context, batch 8 = ~24 GB
Activation Buffer	Dependent on model architecture & sequence length	Intermediate activations = ~8 GB
Total Production Min	Sum of Weights + KV Cache + Activations + 20% Overhead	Recommended GPU Pool: 2x A100 (80GB)

MODULE 5

Infrastructure, Serving & Orchestration

High-throughput inference engines, memory optimization, and cluster sizing.

| High-Performance Inference Engines



vLLM

Open-source serving engine featuring PagedAttention. Dynamically manages KV cache memory to deliver up to 24x higher throughput.



TensorRT-LLM

NVIDIA's specialized compiler and runtime engine. Offers optimized GPU kernel fusion, multi-GPU tensor parallelism, and FP8 support.



TGI & Ollama

Hugging Face Text Generation Inference (TGI) for robust enterprise API serving, and Ollama for lightweight local desktop/developer deployments.

| Advanced Serving Optimization

-  **PagedAttention:** Eliminates memory fragmentation by storing KV cache in non-contiguous virtual memory blocks, allowing near 100% GPU memory utilization.
-  **Continuous Batching:** Iteratively schedules incoming requests at the token level rather than waiting for sequence-level batch completion.
-  **Speculative Decoding:** Uses a small draft model to generate candidate tokens quickly, validated in parallel by the target model in a single forward pass.

Hardware Selection & Datacenter GPUs

Production GPU Tiering

- 🔌 **NVIDIA H100 / H200:** Transformer Engine support (FP8), 80GB-141GB HBM3 VRAM, 3.3TB/s bandwidth. Target for heavy 70B+ inference.
- ⚙️ **NVIDIA A100 / L40S:** Standard workhorse for 8B–70B model serving and fine-tuning workloads.
- 💻 **Consumer Hardware (RTX 4090 / Mac Studio):** Suitable for development prototyping and INT4 local edge inference.



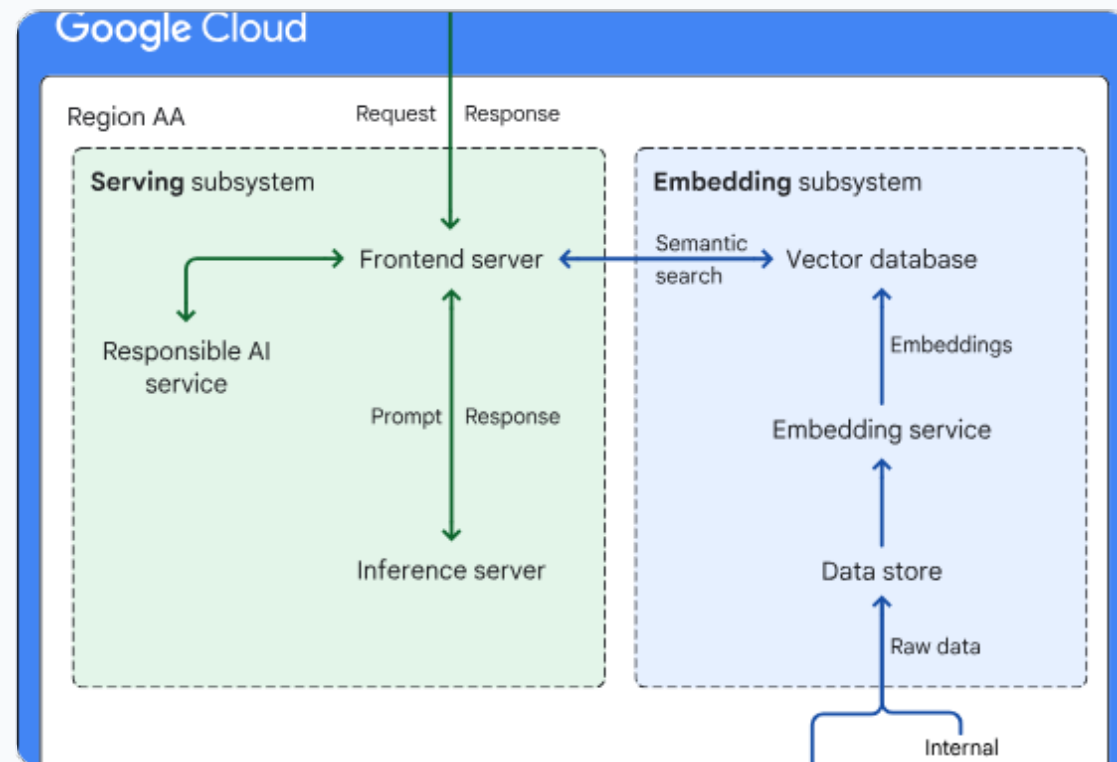
Enterprise Infrastructure Topology

Deployment Topology Selection

Architecting for scale requires balancing multi-region availability with GPU cluster networking bandwidth:

 **Managed Serverless endpoints:** Zero infrastructure overhead, automatic auto-scaling (e.g., Bedrock, Azure OpenAI).

 **Dedicated Kubernetes (K8s) GPU Clusters:** Custom vLLM autoscaling pods behind standard ingress gateways.

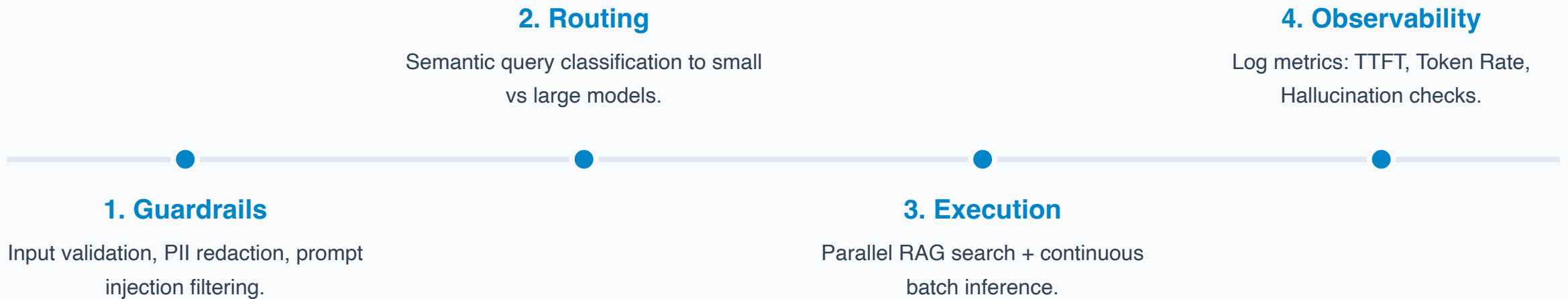


MODULE 6

Enterprise Operations & Security

LLMOps lifecycle, safety guardrails, monitoring, and financial governance.

| The Enterprise LLMOps Lifecycle



Questions & Discussion

Choosing, Adapting & Deploying LLMs — Masterclass Complete

Thank you for participating! Review the documentation & lab code repositories.

| Image Sources



<https://weaviate.io/assets/images/hero-295f13f006733dd2c3564641acac87de.jpg>

Source: weaviate.io



<https://re-cinq.com/cdn-cgi/image/width=1024,quality=75,format=auto/blog-img/llm-architecture-img-1.png>

Source: re-cinq.com



https://miro.medium.com/0*7OaGfO2DctgswevJ.jpeg

Source: pub.towardsai.net



https://giznova.in/wp-content/uploads/2026/03/int8-fp16-int4-inference-execution_pipeline.webp

Source: giznova.in



<https://www.servethehome.com/wp-content/uploads/2024/10/xAI-Colossus-Data-Center-Compute-Hall.jpg>

Source: www.servethehome.com



<https://docs.cloud.google.com/static/architecture/images/rag-capable-gen-ai-app-using-gke-architecture-high-level.svg>

Source: docs.cloud.google.com